

MAN403 - SELECTED TOPICS IN MANAGEMENT SCIENCE

Project Booklet: Eight Companies, Eight Problems

Fall 2026

1 How the projects work

Each of you is assigned **one company** from this booklet and works with it for the whole semester. The company has a decision problem that is hard for two reasons: there are many decisions to make, and some important information is *not known in advance*. You will help the company three times, once with the methods of each block:

- **Project 1** (due Week 6): a method built on a Gurobi model.
- **Project 2** (due Week 10): a method built on dynamic programming.
- **Project 3** (due Week 14): a method that learns from simulated experience.

Because the future is uncertain, what you build is not a single plan but a **decision rule**: something that tells the company what to do *today*, given what it knows *today*.

1.1 You choose the numbers

The stories below fix the *size* of each problem (how many stores, products, machines, ...), so that all eight problems are about equally difficult. **The numbers are yours to choose**: average demands, costs, capacities, distances, and so on. Choose values that make sense for the company, and explain in your report where they come from (a web search, a textbook example, a conversation with someone in the sector, or common sense).

For distances, draw the locations on a simple map, for example on squared paper, and read the distances from it. You can also write a distance table yourself.

1.2 Describing what is uncertain

Each story says what the company does *not* know in advance. The simplest way to describe an uncertain quantity is a small table of possible values and their chances:

Daily sales at a store	quiet day	normal day	busy day
units sold	8	12	18
probability	0.25	0.50	0.25

In Python, one day's sales can then be drawn with one line:

```
sales = random.choices([8, 12, 18], weights=[0.25, 0.50, 0.25])[0]
```

If you prefer, you may use a known probability distribution instead (for example the Poisson distribution for the number of customers). Either way, explain your choice in the report.

1.3 Testing your method fairly

- Build a small **simulator**: a Python function that plays your company's problem day by day (or week by week), uses your decision rule, draws the uncertain values, and adds up the costs.
- **Run the simulator 100 times** and take the average. One *run* plays your company's whole problem once, with its own draw of the uncertain values.
- Use the **same 100 runs** for every method you build: before run k ($k = 1, \dots, 100$), write `random.seed(k)`. Then all your methods face exactly the same "bad days" and "good days", and the comparison between them is fair.
- While you are still trying things out or tuning a method, use other seeds (for example 1000 and above), not the seeds you report with.

1.4 About the reading lists

Each story ends with a short reading list. These are **surveys and problem descriptions**: they explain where the problem comes from, why it is hard, and what people have tried. Read them for understanding, and cite everything you use.

1.5 About using AI

You may use AI tools, and most of you will. The rule of this course is simple: **you direct the tool, the tool does not direct you**. That means you must be able to

- explain, line by line, what your code does and why;
- **change** it when it is wrong — and it will be wrong, because your company, your numbers and your rules are yours alone;
- say what you tried, what failed, and what you changed as a result.

In every project I will ask you to modify your own method in front of me: add a constraint, change a cost, or make the rule behave differently in one situation. A student who can do that has learned the course. A student who can only paste what a tool produced cannot, and the grade will show it.

1.6 Start with the simple rule

Every story ends with a **simple starting rule**: the kind of rule a company might use today without any analysis. Program it first. It shows that your simulator works, and every method you build is judged by how much it improves on this rule.

1.7 What you submit for each project

1. Your Python code. Plain Python is enough: lists, dictionaries, **for** loops and **if** statements. Clear code matters more than clever code. Project 1 also uses Gurobi, as in MAN305.
2. A short report in the style of an academic paper: the company and your numbers, your method explained in words, your results, what you learned, and references.
3. The **results table** below. Use the same table in all three projects.

Comparing with your classmates. Everyone has a different company and everyone chooses their own numbers, so the *costs* in your tables cannot be compared with each other: 240 in one project and 1 800 in another mean nothing side by side. What *is* comparable is the last two columns — **how much a method improved on its own simple rule**, and **what it cost in computing time**. Those two numbers, collected across the class, are what the final report discusses.

Method	Average cost over the 100 runs	Highest cost among the 100 runs	Improvement over the simple rule (%)	Computing time
Simple starting rule			0	
Your method				

P1. Glorfindel: keeping the petrol stations supplied

Glorfindel distributes fuel to **15 petrol stations** from its depot with **one tanker**. Every station has a tank of limited size. How much fuel a station sells in a day is not known in advance: some days are quiet, some are busy. Every morning Glorfindel's planner looks at how much fuel is left at each station and decides which stations the tanker visits today, how much it delivers to each, and in which order it drives. The tanker makes one trip a day.

If a station runs dry, the customers go to a competitor and the sale is lost. Keeping tanks full is not free either: fuel in the tanks is money waiting, and the tanker's kilometres cost money. The planner plans **10 days** ahead.

You choose: the station locations (and so the distances), each station's tank size and average daily sales, the tanker's capacity, the cost per kilometre, the cost of keeping fuel in a tank, and the cost of a lost sale.

Uncertain: each station's daily sales.

Goal: the lowest total cost of driving, keeping stock, and lost sales over the 10 days.

Simple starting rule: "Every morning, fill up every station that has less than one day of sales left. If the tanker cannot carry enough for all of them, serve the emptiest ones first. Always drive to the nearest station not yet visited."

Further reading: [3].

P2. Fossoglor: what to produce this week

Fossoglor Industries makes **8 products** on **one production line**. Before the line can make a product, it must be set up for it: a setup takes some hours and costs money. The line has a limited number of working hours per week. How much of each product customers will order next week is not known in advance.

Every Monday the production planner decides which products to set up this week and how many units of each to make. Making large batches saves setups but fills the warehouse; making small batches keeps stock low but wastes hours on setups. If a customer order cannot be delivered from stock, it waits until the product is made, and Fossoglor pays a penalty for every week it waits. The planner plans **12 weeks** ahead.

You choose: each product's average weekly demand, production time per unit, setup time and setup cost, the weekly working hours of the line, the cost of keeping a unit in stock for a week, and the penalty for a late order.

Uncertain: each product's weekly demand.

Goal: the lowest total cost of setups, stock and late orders over the 12 weeks.

Simple starting rule: "Every Monday, look at the products with less than one week of stock. Starting with the one that has the least stock, produce enough for three weeks, as long as the line's hours allow."

Further reading: [15].

P3. Milkboy: fresh milk before it expires

Milkboy is a dairy that supplies fresh milk to **12 supermarkets** with **two refrigerated trucks**. Milk can be sold for only **three days**; after that it is thrown away. Supermarkets always sell the oldest milk first. How much milk a supermarket sells in a day is not known in advance.

Every morning Milkboy decides which supermarkets each truck visits and how much fresh milk it delivers. Too little milk means empty shelves and lost sales; too much milk means milk that expires and is wasted. The trucks' kilometres cost money too. Milkboy plans **14 days** ahead.

You choose: the supermarket locations, each supermarket's average daily sales and shelf space, the trucks' capacity, the cost per kilometre, the cost of a lost sale, and the cost of a wasted bottle.

Uncertain: each supermarket's daily sales.

Goal: the lowest total cost of driving, lost sales and waste over the 14 days.

Simple starting rule: "Every morning, visit every supermarket that has less than one day of milk, and bring it up to two days of milk. Split these supermarkets between the two trucks by area (for example east and west), and drive to the nearest one each time."

Further reading: [9, 3].

P4. Oronar: a four-stage supply chain

Oronar's product reaches customers through **four stages**: Oronar's factory supplies a distributor, the distributor supplies a wholesaler, and the wholesaler supplies a retail store, where customers buy. Every week each stage orders from the stage above it. An order takes **two weeks** to arrive, and producing at the factory also takes two weeks. How much customers will buy at the store is not known in advance.

Oronar's central planner sees the stock at all four stages and decides every week how much each stage orders. Too much stock anywhere costs money. If the store runs out, customers must wait, and every waiting customer costs a penalty. Placing an order also has a fixed cost (paperwork, a truck). The planner plans **one year** (52 weeks) ahead.

You choose: the average weekly customer demand, the cost of keeping a unit in stock at each stage (usually lower upstream), the penalty for a waiting customer, the fixed cost of an order, and the starting stock at each stage.

Uncertain: weekly customer demand at the store.

Goal: the lowest total cost of stock, waiting customers and orders over the year.

Simple starting rule: "Every week, each stage orders enough to bring its stock (including what is already on the way) back to a fixed target level, for example three weeks of average demand."

Further reading: [2, 13]. The famous "beer game" is exactly this kind of chain.

P5. Findel: collecting empty crates

Findel is a beverage company. Every day one truck visits **30 shops** to collect empty returnable crates. How many crates a shop has is only known when the truck arrives. The truck has limited space: when it is full, it must drive back to the depot, unload, and continue from where it left off.

After each shop, the driver decides where to go next: to another shop, or back to the depot to unload *before* the truck is completely full, if that is likely to save kilometres later (for example, when the depot is nearby). Findel wants to drive as few kilometres as possible.

You choose: the shop locations, each shop's average number of crates, and the truck's capacity (it should need a few trips to the depot on a normal day).

Uncertain: the number of crates at each shop.

Goal: the fewest total kilometres to collect all crates.

Simple starting rule: "Plan a tour by always going to the nearest shop not yet visited. Follow it, and go back to the depot only when the truck is full."

Further reading: [4, 1].

P6. Ronar: technicians on time

Ronar is a technical service company. Tomorrow its **3 technicians** must visit **25 customers**. Every customer has been promised a one-hour visit window, for example 10:00–11:00. Traffic makes travel times uncertain: the same trip can take 20 minutes one day and 35 the next.

In the morning, Ronar decides which customers each technician visits. During the day, after each visit, a technician may change the order of the remaining visits depending on how late or early he or she is running. Arriving before or after the promised window annoys the customer, so it costs a penalty per minute. Technicians working past the end of the shift are paid overtime.

You choose: the customer locations, the visit windows, the average travel times, the service time per visit, the length of the shift, and the penalties for early arrival, late arrival and overtime.

Uncertain: the travel time of every trip.

Goal: the lowest total cost of travel, early and late arrivals, and overtime.

Simple starting rule: “Divide the customers among the technicians by area. Each technician visits his or her customers in the order of their visit windows and never changes the plan.”

Further reading: [6, 4].

P7. Glor: the print shop’s next job

Glor is a print shop with **one printing machine**. Orders arrive during the **8-hour working day**: when a customer brings an order, Glor learns how long it will take to print, when the customer wants it, and how important the customer is. Nobody knows in advance when the next order will arrive or what it will be.

Whenever the machine finishes a job, Glor chooses which waiting order to print next. Once started, a job is not interrupted. Every order that is finished after its promised time costs a penalty for each minute of delay, and important customers cost more.

You choose: how many orders arrive on an average day, the range of printing times, how promised times are set, and the importance levels of customers.

Uncertain: when orders arrive and what they are.

Goal: the lowest total delay penalty, with important customers counting more.

Simple starting rule: “Always print next the waiting order with the earliest promised time.”

Further reading: [10]. A well-known dispatching rule you may use as a second benchmark: the ATC rule of [16].

P8. Fossoronar: forty jobs, four machines

Fossoronar is a workshop that must finish **40 jobs** on **4 machines**. Two of the machines are new and faster than the others. Every job has a due date and an importance level. Fossoronar has an estimate of how long each job takes, but the real time is only known when the job is finished: some jobs go smoothly, some run into problems.

At the start of the day four jobs are started, one on each machine. After that, whenever a machine becomes free, Fossoronar chooses which remaining job to start on it. Every job finished after its due date costs a penalty for each minute of delay, and important jobs cost more.

You choose: the jobs’ estimated times, due dates and importance levels, and how much faster the new machines are.

Uncertain: how long each job really takes.

Goal: the lowest total delay penalty.

Simple starting rule: “Always start next the remaining job with the earliest due date. If several machines are free, use the fastest one.”

Further reading: [10].

Reading on the methods

Block 1: [8, 12]. Block 2: [7, 5, 11]. Block 3: [14, 11].

These references are starting points. Read what you cite, and cite everything you use.

References

- [1] Bertsimas, D. J. (1992). A vehicle routing problem with stochastic demand. *Operations Research*, 40(3), 574–585.
- [2] Clark, A. J. & Scarf, H. (1960). Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4), 475–490.

- [3] Coelho, L. C., Cordeau, J.-F. & Laporte, G. (2014). Thirty years of inventory routing. *Transportation Science*, 48(1), 1–19.
- [4] Gendreau, M., Laporte, G. & Séguin, R. (1996). Stochastic vehicle routing. *European Journal of Operational Research*, 88(1), 3–12.
- [5] Gromicho, J., van Hoorn, J. J., Kok, A. L. & Schutten, J. M. J. (2012). Restricted dynamic programming: a flexible framework for solving realistic VRPs. *Computers & Operations Research*, 39(5), 902–909.
- [6] Kenyon, A. S. & Morton, D. P. (2003). Stochastic vehicle routing with random travel times. *Transportation Science*, 37(1), 69–82.
- [7] Malandraki, C. & Dial, R. B. (1996). A restricted dynamic programming heuristic algorithm for the time dependent traveling salesman problem. *European Journal of Operational Research*, 90(1), 45–55.
- [8] Maniezzo, V., Stützle, T. & Voß, S. (Eds.) (2010). *Matheuristics: Hybridizing Metaheuristics and Mathematical Programming*. New York: Springer.
- [9] Nahmias, S. (1982). Perishable inventory theory: a review. *Operations Research*, 30(4), 680–708.
- [10] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Cham: Springer.
- [11] Powell, W. B. (2011). *Approximate Dynamic Programming: Solving the Curses of Dimensionality* (2nd ed.). Hoboken, NJ: Wiley.
- [12] Ropke, S. & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.
- [13] Serman, J. D. (1989). Modeling managerial behavior: misperceptions of feedback in a dynamic decision making experiment. *Management Science*, 35(3), 321–339.
- [14] Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.
- [15] Tempelmeier, H. (2013). Stochastic lot sizing problems. In J. M. Smith & B. Tan (Eds.), *Handbook of Stochastic Models and Analysis of Manufacturing System Operations*. New York: Springer.
- [16] Vepsäläinen, A. P. J. & Morton, T. E. (1987). Priority rules for job shops with weighted tardiness costs. *Management Science*, 33(8), 1035–1047.